

Contest Platform with Live Leaderboard

Database Implementation & Core Functionalities (MongoDB + Redis)

Team	Upayan Mazumder (24BDS0367) · Aditya Rawat (24BCE2992) · Shloak Sinha (24BDS0378)
Document	Review 2 Report — implementation follow-on to the Review 1 design (attached separately as <code>DBMS_Review_1_Contest_Platform.pdf</code>)
Stack	Python 3.11 + FastAPI + PyMongo · MongoDB 7 (single-node replica set, for transactions) · Redis 7
Repository	Local git repository at the project workspace (<code>rankstack/</code>), 13 commits, clean working tree, no secrets committed — see §8

01 Rubric Mapping & Summary

Criterion	Marks	What was delivered
Database implementation (collections, relationships, sample data)	2 / 2	5 MongoDB collections with real ObjectId references, <code>\$jsonSchema</code> validators on every collection, 88 referentially-consistent sample records loaded via a programmatic seed script.
CRUD operations completed	3 / 3	Full create/read/update/delete for all 5 collections, exposed as a FastAPI REST surface and verified against a live running server (curl transcripts in §3).
Advanced NoSQL features (aggregation, indexing, transactions, etc.)	5 / 5	5 aggregation pipelines, 6 indexes (incl. a compound index, <code>.explain()</code> -verified), 3 real multi-document transactions on a replica set, and a polymorphic <code>oneOf \$jsonSchema</code> validator. Redis (leaderboard/session/rate-limit) implemented as a stretch, per the Review 1 design.

02 Database Implementation

2.1 Collections & relationships

The five collections from the Review 1 design were implemented as-is in MongoDB, with every relationship expressed as an ObjectId reference (never embedded, except `contests.participants` , which stays a small array of refs since it is always read alongside its parent contest).

users	teams	contests	problems
_id, name, email (unique idx), passwordHash, role, totalScore, teamIds[], createdAt	_id, name, memberIds[], totalScore, createdAt	_id, title, description, startTime, endTime, status, createdBy, problemIds[], participants[], createdAt	_id, contestId, type, title, difficulty, points, attemptCount, + type- specific fields, createdAt
submissions			
_id, contestId, problemId, submittedBy{refType,refId}, answer, status, score, submittedAt			

Relationships implemented: User → Contest (embedded ref array), Contest → Problems (`problems.contestId`), User/Team → Submission (`submittedBy`), Submission → Problem (`problemId`), Team → Users (`memberIds`), and a User → Team back-reference (`users.teamIds`) added for the team-membership transaction in §4.3.

2.2 Schema validation (\$jsonSchema)

Every collection is created with a MongoDB `$jsonSchema` validator (`app/db.py`). The `problems` collection additionally uses a `oneOf` clause so the database itself enforces different required fields per polymorphic `type` — a direct NoSQL-level expression of the Review 1 polymorphic design:

```
# app/db.py – problems validator (excerpt)
"oneOf": [
  {"properties": {"type": {"enum": ["mcq"]}, "options": {...}, "correctAnswer": {...}},
   "required": ["options", "correctAnswer"]},
  {"properties": {"type": {"enum": ["coding"]}, "testCases": {...}},
   "required": ["testCases"]},
  {"properties": {"type": {"enum": ["subjective"]}, "evaluationRubric": {...}},
   "required": ["evaluationRubric"]}
]
```

LIVE VERIFICATION

Inserting an `mcq` document missing `options / correctAnswer` → `pymongo.errors.WriteError` (rejected by MongoDB itself). Inserting the same document with those fields present → accepted. Confirmed against all 5 collections.

2.3 Sample data loaded

`scripts/seed.py` programmatically generates a referentially-consistent dataset (no hand-typed records; deterministic via a fixed `Faker / random` seed), comfortably above the 50-record minimum:

Collection	Count	Notes
users	15	2 admin, 13 participant; passwords PBKDF2-hashed
teams	5	3 members each, drawn from the 15 users
contests	5	2 ended, 1 live, 2 upcoming (status computed from real timestamps)
problems	15	3 per contest — 5 mcq, 5 coding, 5 subjective

submissions	48	only for live/ended contests (upcoming contests realistically have none yet)
Total	88	75% above the 50-record minimum

LIVE OUTPUT — MAKE SEED

```
Seed complete. Record counts:
  users: 15
  teams: 5
  contests: 5
  problems: 15
  submissions: 48
  TOTAL: 88
```

Referential integrity was checked programmatically after seeding: every `submissions.contestId / problemId / submittedBy.refId`, every `problems.contestId`, every `teams.memberIds` entry, and every `contests.problemIds / participants` entry resolves to a real document in the target collection — **0 violations** found.

03 CRUD Operations

All five collections expose a full REST CRUD surface via FastAPI (`app/api/`), running against the live MongoDB/Redis stack (not mocked). Swagger UI is auto-served at `/docs` for interactive testing.

Collection	Routes
users	POST <code>/users</code> , GET <code>/users</code> , GET <code>/users/{id}</code> , PATCH <code>/users/{id}</code> , DELETE <code>/users/{id}</code>
teams	POST <code>/teams</code> , GET <code>/teams</code> , GET <code>/teams/{id}</code> , PATCH <code>/teams/{id}</code> , POST <code>/teams/{id}/members</code> , DELETE <code>/teams/{id}/members/{userId}</code> , DELETE <code>/teams/{id}</code>
contests	POST <code>/contests</code> , GET <code>/contests</code> , GET <code>/contests/{id}</code> , PATCH <code>/contests/{id}</code> , PATCH <code>/contests/{id}/status</code> , POST <code>/contests/{id}/participants</code> , GET <code>/contests/{id}/leaderboard</code> , DELETE <code>/contests/{id}</code>
problems	POST <code>/problems</code> (polymorphic), GET <code>/problems?contestId=</code> , GET <code>/problems/{id}</code> , PATCH <code>/problems/{id}</code> , DELETE <code>/problems/{id}</code>
submissions	POST <code>/submissions</code> , GET <code>/submissions?contestId=&userId=&status=</code> , GET <code>/submissions/{id}</code> , PATCH <code>/submissions/{id}/status</code> , DELETE <code>/submissions/{id}</code>

LIVE EVIDENCE — FULL CRUD ROUND-TRIP AGAINST A RUNNING UVICORN SERVER (CURL)

```
$ curl -X POST http://localhost:8000/users -d '{"name":"Doc Evidence User","email":"doc.evidence2@rankstack.io","password":"secret123"}'
{"id":"6ab3f9a580961206777c15a1","name":"Doc Evidence User","email":"doc.evidence2@rankstack.io","role":"participant","totalScore":0.0,"teamIds":[],"createdAt":"2026-09-23T16:09:09.451309Z"}

$ curl http://localhost:8000/users/6ab3f9a580961206777c15a1 # READ
{"id":"6ab3f9a580961206777c15a1", ... "name":"Doc Evidence User", ...}

$ curl -X PATCH .../users/6ab3f9a5... -d '{"name":"Doc Evidence User (Updated)}' # UPDATE
{"id":"6ab3f9a580961206777c15a1", "name":"Doc Evidence User (Updated)", ...}

$ curl -X DELETE .../users/6ab3f9a5... # DELETE
status: 204

$ curl .../users/6ab3f9a5... # confirm gone
status: 404
```

04 Advanced NoSQL Features

4.1 Aggregation pipelines (5)

Implemented in `app/aggregations/pipelines.py`, each exposed as both a plain Python callable and a `GET /aggregations/...` endpoint. Live output against the seeded 88-record dataset:

1. Leaderboard for a contest — group submissions by submitter, sum score, sort desc.

```
[{'totalScore': 40, 'submissionCount': 2, 'refId': '...cd', 'refType': 'user'},
 {'totalScore': 40, 'submissionCount': 2, 'refId': '...d1', 'refType': 'user'},
 {'totalScore': 30, 'submissionCount': 2, 'refId': '...ce', 'refType': 'user'}, ...]
```

2. Average score per problem, grouped by difficulty — two-stage: per-problem avg, then grouped by `$lookup` -ed difficulty.

```
[{'difficulty': 'easy', 'avgScorePerProblem': 3.81, 'problemCount': 3},
 {'difficulty': 'hard', 'avgScorePerProblem': 18.17, 'problemCount': 3},
 {'difficulty': 'medium', 'avgScorePerProblem': 11.94, 'problemCount': 3}]
```

3. Submission count by status, per contest.

```
[{'contestId': '...e1', 'title': 'Expanded static flexibility Contest',
 'statusCounts': [{'status': 'incorrect', 'count': 4}, {'status': 'correct', 'count': 8},
 {'status': 'partial', 'count': 4}]}, ...]
```

4. Distinct problems solved (status = correct) per user, across all contests.

```
[{'userId': '...ce', 'distinctProblemsSolved': 4, 'name': 'Angie Henderson'},
 {'userId': '...d0', 'distinctProblemsSolved': 3, 'name': 'Cristian Santos'}, ...]
```

5. Team performance — `$lookup` from `teams` to `submissions` via `memberIds`, aggregating total score per team.

```
# app/aggregations/pipelines.py — team_performance (excerpt)
{"$lookup": {"from": "submissions", "let": {"memberIds": "$memberIds"},
  "pipeline": [{"$match": {"$expr": {"$and": [
    {"$eq": ["$submittedBy.refType", "user"]},
    {"$in": ["$submittedBy.refId", "$$memberIds"]}]}}}],
  "as": "memberSubmissions"}}
```

```
[{'name': 'Team Lavender Trial', 'totalScore': 195.0, 'submissionCount': 12, 'memberCount': 3},
 {'name': 'Team Darkgreen Husband', 'totalScore': 165.0, 'submissionCount': 18, 'memberCount': 3},
 {'name': 'Team Lightseagreen Long', 'totalScore': 60.0, 'submissionCount': 6, 'memberCount': 3},
 ...]
```

4.2 Indexing

Collection	Index	Keys	Purpose
users	uniq_email	{email: 1} <i>unique</i>	enforce unique registration, fast login lookup
problems	by_contest	{contestId: 1}	"all problems in a contest"
submissions	by_contest_and_submitter	{contestId: 1, submittedBy.refId: 1} <i>compound</i>	leaderboard/submission-history queries
submissions	by_problem	{problemId: 1}	per-problem submission lookups
teams	by_member	{memberIds: 1}	"which teams is this user on"
contests	by_status	{status: 1}	filter by upcoming/live/ended

LIVE EVIDENCE — .EXPLAIN() PROVES THE INDEX IS USED, NOT A COLLECTION SCAN

```
$ db.problems.find({contestId: ObjectId("...")}).explain()
Winning plan stage: FETCH -> IXSCAN
Index used: by_contest
Documents returned: 3

$ db.submissions.find({contestId: ..., "submittedBy.refId": ...}).explain()
Index used: by_contest_and_submitter
```

4.3 Transactions (3, on a single-node replica set)

MongoDB transactions require a replica set — a standalone `mongod` does not support `start_transaction()`. `docker-compose.yml` runs `mongod --replSet rs0` with a one-shot `rs.initiate()` container. All three transactions live in `app/services.py` and are used by both the API and the seed script (so denormalized counters stay consistent with the raw submission data everywhere):

1. **create_submission** — insert a submission + atomically increment `problems.attemptCount`.
2. **update_submission_status** — update a submission's status/score + atomically apply the score delta to the submitter's cached `totalScore` on `users / teams`.
3. **add_team_member / remove_team_member** — update `teams.memberIds` + `users.teamIds` back-reference atomically.

```
# app/services.py – transaction #1 (excerpt)
with client.start_session() as session:
    with session.start_transaction():
        result = db["submissions"].insert_one(doc, session=session)
        db["problems"].update_one({"_id": problem_id},
                                   {"$inc": {"attemptCount": 1}}, session=session)
```

LIVE EVIDENCE — BEFORE/AFTER COUNTERS, VIA SCRIPTS/DEMO.PY

```
problem.attemptCount before=7 after=8 (submission inserted atomically)
user.totalScore      before=90 after=100 (status+score updated atomically)
```

05 Redis Integration

Implemented now rather than deferred to Review 3, per the Review 1 design. Runs as a Docker service alongside MongoDB; three key patterns, all wired into the live API:

Key pattern	Type	Used for
<code>leaderboard:<contestId></code>	Sorted Set	O(log N) live rank reads, kept in sync with every score-changing transaction
<code>session:<sessionId></code>	String + TTL	login sessions, expire on inactivity
<code>rate_limit:<userId></code>	String counter + TTL	submission throttling per time window

LIVE EVIDENCE — RATE LIMITER (MAX 10 SUBMISSIONS / 60S) AGAINST THE RUNNING API

```
$ for i in 1..12: curl -X POST http://localhost:8000/submissions ...
201 201 201 201 201 201 201 201 201 201 429 429
```

06 Intermediate Demonstration

`scripts/demo.py` (`make demo`) is a self-cleaning script a faculty member can run live end-to-end. It connects to the real database and prints, in order: a CRUD round-trip, an indexed query with `.explain()` proof, a multi-document transaction with before/after counters, all 5 aggregation results, and the Redis leaderboard/session/rate-limit state — verified twice end-to-end from a completely clean slate (`docker compose down -v && make up && make init-db && make seed && make demo`), and again against a live `uvicorn` server via `curl` for the CRUD/Redis evidence captured in this document.

07 Source Code Repository

7.1 Layout

```

app/
  main.py          FastAPI app + router wiring
  config.py        Settings (env-driven)
  db.py            Mongo/Redis clients, $jsonSchema validators, indexes
  security.py      Password hashing (PBKDF2), session tokens
  redis_ops.py     Leaderboard / session / rate-limit Redis operations
  services.py      Multi-document transactions (shared by API + seed)
  models/          Pydantic request/response schemas per collection
  api/             FastAPI routers: users, teams, contests, problems, submissions,
sessions
  aggregations/pipelines.py  The 5 aggregation pipelines + their endpoints
scripts/
  init_db.py       create collections/validators/indexes
  seed.py          wipe + reseed ~88 sample records
  demo.py          intermediate demonstration script
docker-compose.yml  MongoDB (replica set) + Redis
Makefile           up / down / init-db / seed / demo / api / reset
README.md, .env.example, .gitignore, requirements.txt

```

7.2 Setup (also in README.md)

```

make up          # start MongoDB (replica set) + Redis via docker compose
make install     # create venv, install dependencies (needs python3.11)
make init-db     # create collections, $jsonSchema validators, indexes
make seed        # load ~88 referentially-consistent sample records
make demo        # run the intermediate demonstration
make api         # serve the REST API at http://localhost:8000/docs

```

7.3 Commit history

13 commits, one logical change each (infra → db/schema → models → security/redis → transactions → seed → api → demo → makefile → docs → a live-testing bugfix), no secrets committed (`.env` / venv gitignored, only `.env.example` tracked):

```

0c815e7 fix: use non-reserved email TLD (rankstack.io) so POST /users accepts seeded/demo addresses
d073377 chore: remove unused empty tests package and dead pytest Makefile target
4c11c55 chore: add httpx dev dependency for TestClient-based smoke tests
475ea77 docs: README with setup, run instructions, rubric mapping
75f58f1 chore: Makefile for up/down/seed/demo/api workflow
1ea9fc1 demo: intermediate walkthrough script (CRUD, indexed query, transaction, aggregations, redis)
2ab9f8a api: FastAPI app + CRUD routers for all 5 collections, sessions, aggregation endpoints
635f751 seed: generate 88 referentially-consistent sample records across all collections
e4071d3 services: multi-document transactions (submission+counter, score+totalScore, team member
back-ref)
92c29f7 core: password hashing, redis leaderboard/session/rate-limit ops, init_db script
23149a8 models: pydantic request/response schemas incl. polymorphic problems
1a21af7 db: mongo/redis clients, $jsonSchema validators, indexes
dae96d2 infra: docker-compose for mongo single-node replica set + redis, base deps

```